

CS201 Winter 2025 — Lecture Notes

Clarissa Littler

Winter 2025

Preface

These are my running notes for the term — one section per class meeting, walking through whatever we covered that day with the actual code we typed up in class. The source files all live in the `lectureN/` subdirectories of the repo, and the snippets I quote here are pulled straight out of those files. If you missed a lecture, want a refresher, or are studying for the final, this is the one-stop overview.

Contents

1	Lecture 1 — Welcome, What an OS Is, and Bits	1
2	Lecture 2 — Sizes, Pointers, Endianness, and IEEE-754	4
3	Lecture 3 — Review, Stacks, and First Assembly	7
4	Lecture 4 — Instructions, Data, Compares, and Loops	9
5	Lecture 5 — Hello, Echo, Labels, and <code>call/ret</code>	12
6	Lecture 6 — Functions with Stack Frames	15
7	Lecture 7 — Forking, Signals, and a Loose End in Assembly	18
8	Lecture 8 — Signals Continued, Threads Up Next	21
9	Lecture 9 — Threads	24
10	Lecture 10 — Mutexes Reviewed and Condition Variables	26
11	Lecture 11 — File I/O in C	29
12	Lecture 12 — Sockets, an Introduction	31
13	Lecture 13 — IPC: Pipes, Named Pipes, and Shared Memory	34

1 Lecture 1 — Welcome, What an OS Is, and Bits

This was the orientation lecture. I sketched out what we're going to spend the term on, made a pitch for why systems programming is interesting, and then we dove straight into the bit-level stuff that underlies everything else.

What this class is

The class is a mix of:

- How operating systems actually work
- How programs run on a real machine
- C programming
- x86-64 assembly
- The contingencies of doing all of this on Linux
- Concurrent and parallel programming

Plus some smaller tangents along the way — a tiny bootloader, some CTF-flavored challenges, me rambling about microkernels, etc.

What is an operating system, anyway?

We took a few minutes to brainstorm. The short version: an OS is the piece of software that mediates between hardware and everything else, and the things you tend to want from it are process management, memory management, file I/O, and a uniform interface to devices.

A quick C tour

Most of you have C++ experience, so I focused on what's different in C. The minimum-viable program is just:

```
#include <stdio.h>

int main(){
    printf("Hello, world!\n");
    return 0;
}
```

A few things to call out coming from C++:

- I/O is `printf/scanf`, not streams. The format specifiers (`%d`, `%s`, `%lu`, ...) are how the function knows what types to expect.
- No pass-by-reference — you pass pointers if you want a callee to modify a caller's variable.
- No `cin`-style line-buffered input convenience; `scanf` reads the format you tell it to.
- `malloc/free` instead of `new/delete`.

`adder.c` drove these home: read two integers and print their sum.

```
int num1; int num2;
printf("Enter a number: ");
scanf("%d",&num1);
printf("Enter another number: ");
scanf("%d",&num2);
printf("So %d and %d added together make %d\n",num1,num2,num1+num2);
```

The `&num1` is the part that matters: `scanf` needs an *address* to store into, because there's no other way to write out-parameters in C.

`echo.c` is similar but for a string; it also shows that an array name is essentially a pointer:

```
char msg[50];
printf("Type somethin' will ya: ");
scanf("%s",msg);
printf("You said: %s\n",msg);
printf("And msg is actually: %p\n",msg);
```

malloc and structs

Two tiny demos. `malloc1.c` is just enough to show that `malloc` takes a byte count and returns a pointer:

```
int* arr = malloc(10000000*sizeof(int));
free(arr);
```

`malloc2.c` adds a struct and shows the two field-access syntaxes:

```
struct Point* pointy = malloc(sizeof(struct Point));
(*pointy).x = 10;
(*pointy).y = 20;
pointy->x = 50; // shorthand for (*pointy).x
pointy->y = 40;
free(pointy);
pointy = 0;
```

The `->` arrow is just sugar for “dereference and access a field.” Always free what you malloc, and zero out the pointer afterward so you don't accidentally use it again.

Bits and bytes

A byte is 8 bits. The bits in a byte read most-significant on the left, so 10010101 unpacked is

$$1 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 149.$$

The bitwise operators in C are:

- `&` — bitwise AND.
- `|` — bitwise OR.
- `^` — bitwise XOR.

- `~` — bitwise NOT.

Don't confuse them with the boolean operators `&&` and `||`, which short-circuit and produce 0/1.

`bit1.c` is the first “play with bits” program: read an integer, print all 32 bits of it, ask which bit to flip, flip it, loop forever:

```
void printBits(int n){
    for(int i = 31; i >= 0; i--){
        printf("%d", (n >> i) & 1);
    }
    printf("\n");
}

void flipBit(int* n, int c){
    *n = (*n) ^ (1 << c);
}
```

The XOR trick is the key thing: XOR-ing with 1 flips a bit, XOR-ing with 0 leaves it alone. So `n ^ (1 << c)` flips exactly the *c*th bit.

`bitshiftDemo.c` is the most boring file in the directory but its one line —

```
printf("%d\n", 1024 >> 2);
```

— captures something I want you to internalize: shifting right by *k* divides by 2^k (for non-negative integers), and shifting left by *k* multiplies by 2^k .

Two's complement

If we have *n* bits we can represent unsigned 0 through $2^n - 1$ with the usual binary expansion. To represent signed integers, we flip the sign of the top bit's weight:

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i.$$

So 11111111 as a signed 8-bit number is $-128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -1$. Negating a number is “flip all the bits and add 1,” or equivalently “subtract 1 then flip all the bits.”

We'll come back to this next time and play with the actual representations of negative numbers and floats.

2 Lecture 2 — Sizes, Pointers, Endianness, and IEEE-754

Today we made data types concrete: how big they are, what their addresses look like, how a multi-byte value is stored on this machine, and finally how floats work.

Sizes

`size1.c` is just `sizeof` on the basic types:

```
printf("The size of an int is: %lu\n",sizeof(int));
printf("The size of a char is: %lu\n",sizeof(char));
printf("The size of a float is: %lu\n",sizeof(float));
printf("The size of a double is: %lu\n",sizeof(double));
printf("The size of a long int is: %lu\n",sizeof(long int unsigned));
```

On our 64-bit Linux boxes: int = 4, char = 1, float = 4, double = 8, long = 8.

Is char signed?

charSigned.c settles a perennial question:

```
char test = -1;
if(test > 0){
    printf("Well, char is unsigned!\n");
}
else{
    printf("char is signed, I guess\n");
}
```

The C standard does not pin down whether plain `char` is signed or unsigned — it’s *implementation-defined*. On x86-64 GCC it’s signed, so this prints “char is signed, I guess.” If you want to be sure, write `signed char` or `unsigned char`.

Pointer arithmetic

pointArith.c prints the addresses of consecutive elements of two arrays of different types:

```
int arr1[5] = {0,1,2,3,4};
double arr2[5] = {0,1,2,3,4};
for(int i = 1; i < 5; i++){
    printf("The %dth element of arr1 is at: %p\n",i,(void*)(arr1 + i));
    printf("The %dth element of arr2 is at: %p\n",i,(void*)(arr2 + i));
}
```

The takeaway is that pointer arithmetic scales with the size of the *pointee*. Adding 1 to an `int*` advances the address by 4 bytes; adding 1 to a `double*` advances by 8. The compiler quietly multiplies your index by `sizeof(*p)` on every `p + i`.

Endianness

whichEnd.c pokes at how a multi-byte integer is actually laid out in memory:

```
int num = 0x12345678;
unsigned char* pointy = (unsigned char*)&num;
printf("Our number's bytes, in order, are: ");
for(size_t i = 0; i < sizeof(num); i++){
    printf(" %x ",*(pointy+i));
}
```

The cast to `unsigned char*` is the trick: now pointer arithmetic moves us one byte at a time, so we can walk the actual storage. On x86-64 you’ll see the bytes come out as 78 56 34 12 —

reversed from how we wrote the literal. That's little-endian: least-significant byte first. Big-endian machines would print it in the order it was written. Network byte order is big-endian, which is why `htons` and friends exist.

IEEE-754

I wrote out the 32-bit float layout on the board:

- 1 bit sign
- 8 bits exponent (biased by 127)
- 23 bits significand (the “mantissa”)

For normalized numbers the value is

$$(-1)^s \cdot 2^{|e|-127} \cdot (1 + |m|),$$

where the significand is read as a sum of *negative* powers of 2: bit position k after the implicit point contributes 2^{-k} .

A worked example: 0 10000001 1100...0 is

$$(-1)^0 \cdot 2^{129-127} \cdot (1 + \frac{1}{2} + \frac{1}{4}) = 4 \cdot \frac{7}{4} = 7.$$

The reason there's an implicit leading 1 is that the significand is required to be ≥ 1 and < 2 in the normalized case, so we save a bit by not storing it.

The edge cases are handled by the exponent:

- Exponent all 0s, significand 0 $\Rightarrow \pm 0$.
- Exponent all 0s, significand $\neq 0 \Rightarrow$ subnormal: $(-1)^s \cdot 2^{-126} \cdot |m|$ (no implicit 1).
- Exponent all 1s, significand 0 $\Rightarrow \pm \infty$.
- Exponent all 1s, significand $\neq 0 \Rightarrow \text{NaN}$.

`floatMunger.c` is just `bit1.c` from last time, but the integer-bit-flipper is run on a `float` via a pointer cast:

```
float num;
scanf("%f",&num);
while(1){
    printf("The number is: %.40f\n",num);
    printBits((int*)&num);
    printf("Which bit to flip?: ");
    scanf("%d",&choice);
    flipBit((int*)&num,choice);
}
```

This is the single best way to internalize the format: type a familiar number, see its bits, flip the sign bit, watch it negate; flip the low exponent bit, watch it double.

Where floats lie to you

floatTest.c shows the canonical disappointment:

```
double num1 = 0;
double num2 = 0.001;
for(int i = 0; i < 10000; i++){
    num1 = num1 + num2;
}
if(num1 == 10){
    printf("Yee-caw!\n");
}
else {
    printf("Whoops!\n");
    printf("num1 is really: %.20f\n", num1);
}
```

0.001 is not exactly representable in binary floating point. The error is small per addition, but we accumulate it 10,000 times, and the final answer is not 10. This is the kind of thing where “check floats with ==” will burn you forever.

The deeper reason it’s unavoidable: there are uncountably many real numbers but only finitely many floats, so we’re forced to round most reals to the nearest representable one. IEEE-754 specifies “round-to-even” on ties, so the rounding doesn’t bias one direction on average.

Posits, briefly

I previewed posits because they’re the topic of the next discussion and they’re the next-day reading. The short version: they’re an alternative number format with variable-width fields, parameterized by total bit width n and max exponent bits es . They give up some precision at the extremes in exchange for more precision in the middle of the number line, and they have just one “not-a-real” sentinel instead of IEEE’s menagerie. The tutorial in positTutorial.org walks through the encoding in detail.

3 Lecture 3 — Review, Stacks, and First Assembly

Today was the pivot from C to assembly. We reviewed IEEE and posits, talked about the upcoming stack assignment, and then dove into the GNU assembler.

IEEE-754 review

I worked one more example on the board to nail down the formula. For the normalized 32-bit case,

$$(-1)^{|s|} \cdot 2^{|e|-127} \cdot (1 + |m|).$$

Take 1 00111111 10100...0: sign 1, exponent 63 so $|e| - 127 = -64$, significand $1/2 + 1/8 = 5/8$, so the value is $-1 \cdot 2^{-64} \cdot (13/8)$, a tiny negative number.

I also pointed out a subtlety: if we allowed the significand to be ≥ 2 , a single value would have multiple representations (shift the significand right and bump the exponent). Forcing the implicit leading 1 makes the encoding unique.

Posits, more detail

Same idea as last time but written out more carefully. With $n = 8$, $es = 2$, the bits split as: 1 sign bit, then *regime* bits counted by run-length, then up to es exponent bits, then whatever's left for the significand. The value is

$$(-1)^s \cdot 2^{2^{es} \cdot k} \cdot 2^{|e|} \cdot (1 + |m|),$$

where k is determined by the regime: if the regime starts with 1 and has length r , then $k = r - 1$; if it starts with 0, then $k = -r$. Worked example: 01101101 parses as

$$1 \cdot 2^{2^2} \cdot 2^3 \cdot (1 + 1/4) = 16 \cdot 8 \cdot 5/4 = 160.$$

If you want to see the full picture — including the “not-a-real” sentinel and how to do arithmetic — read positTutorial.org in the repo.

The stack assignment

Reminder: a stack is a LIFO data structure with two operations, **push** and **pop**. For the assignment I want you to implement one in C: pick either a linked list or an array-with-length representation, expose

```
void push(struct Stack* s, int x);
int pop(struct Stack* s);
```

and write a tiny driver that demonstrates LIFO order.

Compiling to assembly: gcc -S

The first thing we did with assembly was to look at what GCC generates for our `hello.c`. `gcc -S hello.c` dumps `hello.s`:

```
.file "hello.c"
.intel_syntax noprefix
.text
.section .rodata
.LC0:
.string "Hello CS201!"
.text
.globl main
.type main, @function
main:
    endbr64
    push rbp
    mov rbp, rsp
    lea rax, .LC0[rip]
    mov rdi, rax
    call puts@PLT
    mov eax, 0
    pop rbp
    ret
```

A few things to notice:

- There are *sections*: `.text` for code, `.rodata` for read-only data (the string lives there).
- GCC swapped our `printf` for `puts`, because the format string had no format specifiers — a tiny optimization.
- This file is in Intel syntax (`.intel_syntax noprefix`); the GNU assembler defaults to *AT&T* syntax, which is what we'll use the rest of the term. PSU does AT&T, PCC does AT&T, that's the convention.

The big visible difference between the two syntaxes is operand order: in AT&T, `mov S, D` means “move *S* to *D*” (source first); in Intel, `mov D, S` means the same thing (destination first). Register names get a `%` prefix in AT&T, immediates get a `$` prefix.

Our first hand-written assembly: `ret1.s`

```
.text
.global _start

_start:
    mov $60,%rax
    mov $-1,%rdi
    syscall
```

This is the smallest assembly program that does something observable. `_start` is the entry point that the linker expects (no C runtime to call `main` for us). We put 60 in `%rax` (the `syscall` number for `exit` on x86-64 Linux), put our exit status in `%rdi`, and execute the `syscall` instruction. The shell will report the exit status as 255, because exit codes are masked to 8 bits.

To assemble and link this by hand:

```
as -o ret1.o ret1.s
ld -o ret1 ret1.o
```

Run it, then `echo $?`. That's our entire workflow for the next few lectures.

4 Lecture 4 — Instructions, Data, Compares, and Loops

Today we filled out the assembly toolkit: arithmetic instructions with their size suffixes, the `.data` section for globals, the addressing modes for arrays, and `cmp/jmp` for control flow.

Instruction sizes

Most arithmetic instructions come in size-suffixed flavors:

- `q` — quad, 64 bits
- `l` — long, 32 bits
- `w` — word, 16 bits
- `b` — byte, 8 bits

You can usually let the assembler infer the size from the operands, but sometimes you need to be explicit (e.g. when both operands are immediates or when you're operating on memory through a pointer with no register involved).

add and sub

add2.s:

```
_start:
    mov $1,%rax
    mov $2,%rdx
    sub %rax,%rdx # sub s,d -> d = d - s
    mov %rdx,%rdi
    mov $60,%rax
    syscall
```

In AT&T syntax, `add S, D` computes $D = D + S$, and `sub S, D` computes $D = D - S$. The destination is on the right. This will trip you up at some point this term — I guarantee it. The result of $2 - 1$ is 1, which we move into `%rdi` so it becomes the exit status.

cmp and jmp

`cmp S, D` sets the CPU flags based on $D - S$ but discards the result. The conditional jumps then branch on those flags. `cmp1.s`:

```
_start:
    mov $1,%rax
    mov $2,%rbx

    cmp %rbx,%rax # compare as if %rax - %rbx
    jne less
    mov $-1,%rdi
    jmp finish
less:
    mov $10,%rdi
finish:
    mov $60,%rax
    syscall
```

The cheat sheet for the conditional jumps:

- `je / jne` — equal / not equal.
- `j1 / jle` — less / less-or-equal (signed).
- `jg / jge` — greater / greater-or-equal (signed).
- `jb / ja` — below / above (unsigned).

Pay attention to the operand-order quirk: `cmp %rbx,%rax` followed by `j1` branches if $\%rax < \%rbx$ in the AT&T mental model. (I read it as “D fewer than S” to keep myself straight.)

The .data section

To declare globals you put them in `.data`, with directives that pick the size:

- `.byte` — 8 bits
- `.word` — 16 bits
- `.long` — 32 bits
- `.quad` — 64 bits

You can list multiple values: `.long 1,2,3,4`.

`data1.s` loads a single global, increments it, and exits with its value:

```
.section .data
num1: .long 200

.section .text
.global _start

_start:
    mov num1(%rip),%rbx
    addl $10,(%rbx)
    mov num1,%rdi
    mov $60,%rax
    syscall
```

Heads up: this file as written has a couple of subtleties worth flagging. `num1` is a `.long` (32 bits), but the first `mov num1(%rip),%rbx` loads 64 bits into a 64-bit register — the upper half is whatever happens to live next to `num1` in the `.data` section, which is fine here only because the program exits immediately. The cleaner version would use `movl num1(%rip),%ebx` to load the 32-bit value into the 32-bit register `%ebx` (which auto-zeros the upper half). Mixing `.long` with the 64-bit register names is the single most common bug pattern I see all term — watch for it in your own code.

Arrays and addressing modes

`data2.s` is a for-loop that sums an array:

```
.section .data
arr1: .long 1,2,3,4

.section .text
.global _start

_start:
    lea arr1(%rip),%rbx # %rbx = &arr1
    mov $0,%rcx # i = 0
    mov $0,%rax # sum = 0

loopBody:
    cmp $4,%rcx
    jge done
    add (%rbx,%rcx,4),%rax # sum += arr1[i]
    add $1,%rcx
```

```

        jmp loopBody

done:
    mov %rax,%rdi
    mov $60,%rax
    syscall

```

The new piece is the addressing mode `(%rbx,%rcx,4)`, which computes the address `%rbx + %rcx · 4`. The syntax is (base, index, scale) with scale in {1, 2, 4, 8}. This is exactly how a C-style array index compiles, and you’ll see it constantly.

Same gotcha as before: `arr1` is `.long` (4-byte) elements, so the scale is 4 — but `add (%rbx,%rcx,4),%rax` does a 64-bit load into `%rax`, which reads 8 bytes per iteration. The two adjacent `.long` values fit into one 8-byte read, so on the first iteration we add elements 0 and 1 together, on the next iteration elements 1 and 2, etc. The “correct” version would use `addl (%rbx,%rcx,4),%eax` — 32-bit operand size — so each load picks up exactly one array element. Try both and watch the exit status change.

`data3.s` is the same loop, but it factors out the address computation using `lea`:

```

    lea (%rbx,%rcx,4),%rdx # %rdx = &arr1[i]; no dereference
    add (%rdx),%rax # sum += *(%rdx)

```

The mnemonic `lea` is for “load effective address”: it computes the address that the addressing mode would point to and stuffs that into the destination, but does *not* dereference. Useful when you want a pointer to something rather than the thing itself — the assembly equivalent of C’s `&`.

Hello world preview

I sketched the structure of the next program: `.asciz` for a null-terminated string in `.data`, the `message_len = . - message` idiom for getting the length at assembly time, and the `write` syscall (number 1, args `fd / buf / count` in `%rdi / %rsi / %rdx`). We’ll write the full program first thing next class.

5 Lecture 5 — Hello, Echo, Labels, and call/ret

Today we wrote “hello, world” end-to-end, then `echo` end-to-end, then introduced labels and the `call/ret` mechanism that lets us write actual functions.

Reference: the calling convention

Quick reminder of the registers and how arguments get passed:

- Argument registers (in order): `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`.
- Return value: `%rax`.
- Caller-saved: `%rax`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%r8–%r11`.
- Callee-saved: `%rbx`, `%rbp`, `%r12–%r15`.

Syscall arguments use a slightly different layout (4th arg is `%r10` not `%rcx`), but for our purposes the syscalls we use only need three or fewer args, so we don't have to think about it.

Syscall numbers we'll be using:

- 0 — `read(fd, buf, count)`
- 1 — `write(fd, buf, count)`
- 60 — `exit(status)`

And the file descriptors: 0 = `stdin`, 1 = `stdout`, 2 = `stderr`.

Hello, world

hello.s:

```
.section .rodata
msg: .asciz "Hello world!\n"
hellolen = . - msg

.section .text
.globl _start

_start:
    mov $1,%rax # syscall: write
    mov $2,%rdi # fd = 2 (stderr)
    lea msg(%rip),%rsi
    mov $hellolen,%rdx
    syscall

    xor %rdi,%rdi
    mov $60,%rax
    syscall
```

A few points:

- `.asciz` lays down the characters followed by a null terminator.
- `hellolen = . - msg` is the canonical idiom for string length: `.` is the current address, so subtracting `msg` gives the byte length (including the newline and the trailing `\0`).
- I deliberately wrote `fd = 2 (stderr)` here to make the point that `fd 0 / 1 / 2` are conventions, not magic; you can write to any of them. Change it to `$1` for `stdout`.
- `xor %rdi,%rdi` is the idiomatic way to zero a register — shorter encoding than `mov $0,%rdi`.

Echo, with the .bss section

To read input we need somewhere to read *into*. The `.bss` section is for uninitialized, zero-filled storage that costs nothing on disk:

```
.section .bss
buff: .skip 128
```

`.skip 128` reserves 128 bytes at the label `buff`.

`echo.s` reads from `stdin` and writes back what it got:

```
.section .bss
buff: .skip 128

.section .text
.globl _start

_start:
    mov $0,%rax # read
    mov $1,%rdi # stdin
    lea buff(%rip),%rsi
    mov $128,%rdx
    syscall

    mov %rax,%rdx # bytes-read becomes byte count
    mov $1,%rax # write
    lea buff(%rip),%rsi
    mov $0,%rdi # stdout
    syscall

    xor %rdi,%rdi
    mov $60,%rax
    syscall
```

The clever bit is `mov %rax,%rdx` between the two syscalls: `read` returns the number of bytes actually read in `%rax`, and we feed that straight into `write`’s byte-count argument. Otherwise we’d echo the uninitialized tail of the buffer too.

Bug worth flagging: the file as written has the `%rdi` values for read/write swapped — it sets `%rdi = 1` for the `read` (which would mean “read from stdout”!) and `%rdi = 0` for the `write` (which would mean “write to stdin”). On a normal interactive terminal that fd 0 is opened read/write, so it “works” (it reads from your terminal and writes back to it), but the right values are 0 for read and 1 for write. This is the same fd-mixup pattern I’ll keep flagging all term.

Conditional echoes: `echoBroken.s`

`echoBroken.s` branches based on what’s in the buffer:

```
.section .bss
buff: .skip 128

.section .rodata
msg1: .asciz "Was zero\n"
msg1len = . - msg1
msg2: .asciz "Not zero\n"
msg2len = . - msg2

.section .text
.globl _start

_start:
```

```

        lea buff(%rip),%rbx
        cmpb $0,(%rbx)
        je isZero
        jmp notZero

isZero:
        lea msg1(%rip),%rsi
        mov $msg1len,%rdx
        jmp writeLabel

notZero:
        lea msg2(%rip),%rsi
        mov $msg2len,%rdx

writeLabel:
        mov $1,%rax
        mov $0,%rdi
        syscall
        xor %rdi,%rdi
        mov $60,%rax
        syscall

```

The point isn't really the conditional — the “broken” part is that we never actually *read* into `buff`, so it's always zero. (The corresponding `read` block is commented out at the top.) Same `fd-1/fd-0` swap on the write call, too.

The new instruction here is `cmpb $0,(%rbx)` — compare the single byte at address `%rbx` against 0. The `b` suffix is required because there's no register to hint at the size; we need to tell the assembler we mean a byte.

Local labels and `call/ret`

`labelTest.s` introduces two new ideas: numeric local labels (`1:`, `2:`) and the `call/ret` pair.

```

_start:
        call biglabel2
        call biglabel1

1:
        mov $60,%rax
        xor %rdi,%rdi
        syscall

biglabel1:
        jmp 1f

1:
        lea msg1(%rip),%rsi
        mov $msg1len,%rdx
        mov $1,%rax
        mov $0,%rdi
        syscall
        ret

biglabel2:

```

```

        jmp 1f
1:
        lea msg2(%rip),%rsi
        mov $msg2len,%rdx
        mov $1,%rax
        mov $0,%rdi
        syscall
        ret

```

Two new mechanics:

- *Numeric local labels.* `1:` can be reused as many times as you want. `jmp 1f` means “jump to the next `1:` forward,” `jmp 1b` means “jump to the nearest `1:` backward.” Saves you inventing names for one-off jumps inside a function.
- `call/ret`. `call label` pushes the address of the next instruction and jumps to `label`; `ret` pops that address and jumps back. So now we have real subroutines that can be called from multiple places.

The `fd-0` vs `fd-1` mix-up is in this file too. (When I notice these later when teaching from the file, I treat them as “what does this do? what should it do?” moments — they’re more useful as debugging exercises than mistakes to fix silently.)

6 Lecture 6 — Functions with Stack Frames

Today we made functions feel like real functions: they take arguments, they have local variables on the stack, and they return values cleanly. The two files in this lecture both compute something silly so we can focus on the mechanics.

Plan

The arc:

1. Push the old base pointer.
2. Move `%rsp` into `%rbp` so the new frame starts where the stack currently is.
3. Subtract some multiple of 16 from `%rsp` to carve out room for locals.
4. Use `-8(%rbp)`, `-16(%rbp)`, etc. as your local variables.
5. On the way out, restore `%rsp`, pop `%rbp`, and `ret`.

Why a multiple of 16? Because the ABI requires `%rsp` to be 16-byte aligned at the point of every `call`, and some instructions (notably the SSE ones that shuffle 128-bit values) will fault if it isn’t. Better to over-allocate by 8 bytes than to debug a mysterious crash inside a library function.

`fun1.s`: a function without locals

```

        ## int fun(int a, int b){
        ## return 3*(a + b);
        ## }
fun:
        mov $0,%rax

```



```

    add %rdi,%rax
    add %rsi,%rax
    imul $3,%rax # signed multiply
    ret

_start:
    mov $2,%rdi
    mov $3,%rsi
    call fun
    mov %rax,%rdi
    mov $60,%rax
    syscall

```

Two notes:

- Arguments come in via `%rdi`, `%rsi`, the return value goes back in `%rax`.
- `imul` is signed multiply; `mul` is unsigned. Use `imul` unless you know your operands are unsigned.

This function has no locals, so we don't need to set up a stack frame. Result: $3 \cdot (2 + 3) = 15$, so the program exits with status 15.

fun2.s: a function with locals

```

## long fun(long a, long b){
## long c = a + b;
## long d = 2 + b;
## return c + d;
## }

fun:
    push %rbp
    mov %rsp,%rbp
    sub $16,%rsp # room for two quadword locals

    mov %rdi,-8(%rbp) # local c = a
    add %rsi,-8(%rbp) # c += b
    mov %rsi,-16(%rbp) # local d = b
    addq $2,-16(%rbp) # d += 2
    mov -16(%rbp),%rax
    add -8(%rbp),%rax # %rax = d + c

    leave # mov %rbp,%rsp; pop %rbp
    ret

_start:
    mov $3,%rdi
    mov $4,%rsi
    call fun
    mov %rax,%rdi
    mov $60,%rax
    syscall

```

This is the canonical shape of an assembly function from now on. Walk through it once on paper:

- `push %rbp` stashes the caller's base pointer on the stack.
- `mov %rsp,%rbp` says “the new frame starts here.”
- `sub $16,%rsp` reserves 16 bytes of locals; we treat those as two 8-byte slots at `-8(%rbp)` and `-16(%rbp)`.
- `leave` is shorthand for “`mov %rbp,%rsp; pop %rbp`,” undoing the prologue in two cycles.

With `a = 3`, `b = 4`: `c = 7`, `d = 6`, return `c + d = 13`. Exit status 13.

Aside: closures

I left `closure.lisp` in the directory as a tiny aside about why the stack alone isn't enough for some languages:

```
(defun closure-test (y)
  (let ((beep y))
    #'(lambda (x)
        (incf beep)
        (+ x beep)))))
```

The lambda closes over `beep` — it has to keep `beep` alive past the return of `closure-test`. A plain stack frame can't support that, because the frame is gone the moment the function returns. Languages with closures either heap-allocate the closed-over values or use a more elaborate environment representation. That's the entry point to a much longer conversation about runtimes, which we won't dive into in this class — but [assemblyGuide.org](http://assemblyguide.org) in the repo has more on what your C compiler is doing under the hood.

7 Lecture 7 — Forking, Signals, and a Loose End in Assembly

We swing back to C now, with two big topics: `fork()` and signals. There's also one piece of leftover assembly business at the top.

Loose end: `leaQuestion.s`

Someone asked about `lea` after lecture, so here's a tiny file that uses it:

```
.section .data
arr: .long 1,2,3,4

fun1:
    push %rbp
    mov %rsp,%rbp
    sub $16,%rsp
    movl $3,-16(%rbp)
    mov -16(%rbp),%rcx
1:
    cmp $0,%rcx
    jl 2f
    lea arr(%rip),%rbx
    movl $0,(%rbx,%rcx,4)
    dec %rcx
```

```

        jmp 1b
2:
        leave
        ret

_start:
        call fun1
        mov $60,%rax
        mov arr(%rip),%rdi
        syscall

```

This zeros out the four `.long` elements of `arr` counting down from 3, then exits with whatever `arr[0]` is (zero, post-clear). The point of the file is the `(%rbx,%rcx,4)` addressing: base + index * 4, just like in C indexing.

Process memory layout

Before forks, a tour of how Linux lays out a process in memory — text segment at the bottom, then `.rodata` / `.data` / `.bss`, then the heap growing upward, the stack growing downward from a high address, with mmap'd regions in between. The diagrams at <https://simonis.github.io/Memory/> are useful if you want to see the actual addresses.

fork()

`fork()` clones the current process. After it returns, you have two processes, both at the next instruction, both with copies of all your variables.

`pid1.c` just shows you who you are:

```

printf("Hello! My id is %d and my parent's id is %d\n",
       getpid(), getppid());

```

`pid2.c` actually forks:

```

pid_t pid = fork();
if (pid < 0){
    perror("Fork failed");
    return 1;
}
else if (pid == 0){
    printf("Hello from the child process! My PID is %d\n", getpid());
}
else {
    printf("Hello from the parent process! My child's PID is %d\n", pid);
}
printf("This message is printed by both the parent and the child.\n");

```

The convention for the return value of `fork` is:

- negative $\Rightarrow$ failure;
- zero $\Rightarrow$ “you’re the child;”
- positive $\Rightarrow$ “you’re the parent, and this is your child’s PID.”

Caveat for pid2.c: the parent doesn't `wait()` for the child here, so the child can outlive the parent (briefly) and become a “zombie” until reaped by `init`. Always pair your forks with a `wait()` in real code.

pid3.c adds the `wait`:

```
else if(pid == 0){
    printf("Hey, give me a number, will ya?\n");
    int num;
    int success = scanf("%d",&num);
    return success ? 0 : 1;
}
else {
    wait(&childReturn);
    printf("Here's the int returned by childReturn: %d", childReturn);
}
```

But there's a gotcha: `wait`'s status output is not just the exit code — it's a packed integer where the exit code is in the upper byte and the lower bits encode signals etc. So the printed value isn't the child's return value; it's that value shifted left 8 bits.

pid4.c unpacks it manually:

```
wait(&childReturn);
// WEXITSTATUS(int) => (int >> 8) & 255
printf("Here's the int returned by childReturn: %d\n",
       (childReturn >> 8) & 255);
```

noparent.c uses the macro instead of the bit-fiddle:

```
wait(&childReturn);
if(WEXITSTATUS(childReturn) == 0){
    printf("Thanks for being nice to my child!\n");
}
else{
    printf("They've massacred my boy!\n");
}
```

Always use `WEXITSTATUS` (and friends, like `WIFEXITED`, `WTERMSIG`) rather than rolling your own shifts. They handle every corner case the kernel cares about.

Signals

A signal is an asynchronous interrupt delivered to a process. The kernel maintains a per-process table of handlers; when a signal arrives, the kernel calls the corresponding handler. Useful signals: `SIGINT` (Ctrl-C), `SIGTERM` (please die), `SIGKILL` (die now, can't be caught), `SIGUSR1` / `SIGUSR2` (your own use), `SIGSEGV`, `SIGALRM`.

sig1.c catches Ctrl-C:

```
void handler(int sig){
    (void)sig;
    printf("Caught that sigint!\n");
    exit(0);
}
```

```

}

int main(){
    printf("We have a ctrl-c handler here! I.E. SIGINT (%d)\n",SIGINT);
    signal(SIGINT, handler);
    while(1){
        printf("Beep boop\n");
        sleep(1);
    }
}

```

sig2.c makes the handler run multiple times before exiting, with a countdown:

```

volatile sig_atomic_t num = 3;

void handler(int sig){
    (void)sig;
    if(num > 0){
        printf("Caught that sigint!\n");
        printf("Explosion in...%d\n",num);
        num--;
    }
    else{
        printf("Boom!\n");
        exit(0);
    }
}

```

The `volatile sig_atomic_t` matters: the variable is modified asynchronously by the handler, so the compiler can't cache its value in a register, and we need a type that's guaranteed to be updated atomically with respect to signal delivery.

`tag.c` is the one I had the most fun with — a parent and child take turns sending each other signals:

```

volatile sig_atomic_t num = 0;

void handler1(int sig){ /* child */
    (void)sig;
    printf("I've been tagged by my parent %d times\n",num);
    num++;
}

void handler2(int sig){ /* parent */
    (void)sig;
    printf("I've been tagged by my child %d times\n",num);
    num++;
}

int main(){
    signal(SIGUSR1,handler1);
    signal(SIGUSR2,handler2);
    int pid = fork();
    if(pid < 0) return 1;
    else if(pid == 0){

```

```

    while(num < 5){
        kill(getppid(),SIGUSR2);
        sleep(1);
    }
}
else{
    while(num < 5){
        kill(pid,SIGUSR1);
        sleep(1);
    }
    kill(pid,SIGTERM);
    wait(0);
}
}

```

The point here: signal delivery is an inter-process communication primitive, not just a way for the kernel to tell you about errors. Note the `wait(0)` at the end — the parent must reap the child after killing it, otherwise we leak a zombie.

8 Lecture 8 — Signals Continued, Threads Up Next

A short, dense day on signals. We picked up the parent/child fight that I'd promised and looked at signal masking, then started threading at the very end.

Alarms

`alarms.c` demonstrates `SIGALRM`, which the kernel sends after a delay you request:

```

void alarm_handler(int sig){
    (void)sig;
    printf("The bells have been rung!\n");
}

int main(){
    signal(SIGALRM, alarm_handler);
    alarm(5);
    printf("We slumber\n");
    pause(); // sleep until any signal arrives
    printf("We have awoken!\n");
    return 0;
}

```

`pause()` is great for this: it puts the process to sleep until *any* signal arrives, then wakes it up.

The `rand()` fun fact

`randtest.c` just calls `rand()` ten times in a row:

```

for(int i=0;i<10;i++){
    printf("Here's a random number %d\n",rand());
}

```

If you run it twice you'll notice it produces the same numbers each time. `rand()` is a deterministic pseudorandom sequence, seeded from a default value. To get different output you need `srand(time(NULL))` (or something more entropic) at startup.

Signal masking and unmaskable signals

`stopTest.c` tries to block `SIGSTOP`:

```
sigset_t blocks, prevs;
sigemptyset(&blocks);
sigaddset(&blocks, SIGSTOP);
sigprocmask(SIG_BLOCK, &blocks, &prevs);
while(1){
    printf("Here I am!: %d\n", getpid());
    sleep(1);
}
```

The point: `SIGSTOP` (the one Ctrl-Z sends, kind of, but actually that's `SIGTSTP`) and `SIGKILL` can never be blocked, ignored, or caught. The kernel forces them through. Try it and you'll find Ctrl-Z still suspends the program.

`stopTest2.c` makes the same point with `SIGTSTP`, the “terminal stop” signal that Ctrl-Z actually sends:

```
void handler(int sig){
    (void)sig;
    printf("Mwhahaha\n");
}
int main(){
    signal(SIGTSTP, handler);
    while(1){
        printf("Here I am!: %d\n", getpid());
        sleep(1);
    }
}
```

This one *can* be caught — press Ctrl-Z and you get “Mwhahaha” instead of suspension. To kill the program you'll need Ctrl-C (`SIGINT`) or `kill -9` (`SIGKILL`).

Parent/child signal fight

`signalFight.c` is the whole-page program: parent and child take turns sending each other `SIGUSR1`, randomly damaging each other, until one runs out of HP.

```
volatile sig_atomic_t hp = 50;
volatile sig_atomic_t stillFighting = 1;
volatile sig_atomic_t won = 1;

void hitHandler(int sig){
    (void)sig;
    if(hp > 0){
        int d = rand()%5+1;
        char str[50];
```

```

    sprintf(str,"I, # %d, have been hit! I took %d damage!\n",getpid(),d);
    write(STDOUT,str,strlen(str));
    hp = hp - d;
    if(hp <= 0){ stillFighting = 0; won = 0; }
}
}

void fightLoop(int enemy){
    while(stillFighting){
        char str[50];
        sprintf(str,"I, mr. %d, have %d hp left\n",getpid(),hp);
        write(STDOUT,str,strlen(str));
        kill(enemy,SIGUSR1);
        int rest;
        do { rest = sleep(rand()%4+1); } while(rest > 0);
    }
    kill(enemy,SIGUSR2);
}

```

Two crucial details inside the handler:

- It uses `write()` on `STDOUT` rather than `printf`, because `printf` is not async-signal-safe — if a signal interrupts the middle of a `printf` and your handler also calls `printf`, you can deadlock or corrupt output. `write` is safe.
- It does the formatting with `sprintf` into a local buffer first. (`sprintf` isn't strictly safe either, but it's much less risky than `printf` since it doesn't touch the `stdio` lock.)
- The `do/while(rest > 0)` loop around `sleep` handles the case where `sleep` is interrupted by a signal — it returns the unslept time, and we keep going until we've actually slept the full duration.

The seeding “`srand(time(0) ^ getpid())`” is a small trick to keep the parent and child from getting identical random sequences.

There's a commented-out alternative using `sigaction` with `SA_RESTART` at the top of `main` — worth internalizing as the more robust modern API. `signal()` is fine for quick demos but `sigaction` gives you control over exactly which other signals are masked while your handler runs, and lets you get automatic syscall restart.

We'll get into `pthread`s next time.

9 Lecture 9 — Threads

Threads are like processes that share memory. You get most of the parallelism benefit and lose most of the isolation cost — which is also where the danger comes from. Today: how to start them, how to wait for them, how to pass arguments and get results, and the first mutex.

For more depth, `concurrencyGuide.org` in the repo is the canonical reference for everything we're doing this week and next.

The basic shape

A thread is started with `pthread_create`, which takes a thread handle, attribute pointer (usually `NULL`), function pointer, and an opaque `void*` argument:

```
int pthread_create(pthread_t* thread, pthread_attr_t* attr,
                  void* (*func)(void*), void* arg);
```

The thread function takes a `void*` and returns a `void*`. That `void*` is C's polymorphism: the caller casts whatever they want into it on the way in, and the thread casts it back to its real type inside.

`threads1.c`:

```
void* func1(void* arg){
    (void)arg;
    printf("Hi I'm a thread!\n");
    return NULL;
}

int main(){
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, func1, NULL);
    pthread_create(&thread2, NULL, func1, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    return 0;
}
```

`pthread_join` blocks until the named thread finishes — roughly the threading equivalent of `wait`. The second argument is a place to stash the return value, or `NULL` if you don't want it.

If you forget to join your threads, `main` can return out from under them and the whole process gets cleaned up; sometimes this looks like the threads “didn't run.” Always join.

Reading return values

`threads2.c` captures the return value:

```
int* ret1; int* ret2;
pthread_create(&thread1, NULL, func1, NULL);
pthread_create(&thread2, NULL, func1, NULL);
pthread_join(thread1, (void**)&ret1);
pthread_join(thread2, (void**)&ret2);
printf("The value of ret1 is: %p\n", (void*)ret1);
```

The double-pointer is because we want `pthread_join` to write *into* our `ret1`: it needs the address of our pointer.

`threads3.c` actually returns something nontrivial:

```
void* func1(void* arg){
    (void)arg;
    int* sleepPointer = malloc(sizeof(int));
    *sleepPointer = rand()%5+1;
```

```
    sleep(*sleepPointer);
    return sleepPointer;
}
```

Important point: you cannot return a pointer to a stack local from a thread function (or any function). The stack frame is gone the moment the function returns. `malloc` the value if you need it to outlive the thread; the joiner is responsible for **free**-ing it.

Passing arguments in

`threads4.c` uses the argument slot:

```
void* func1(void* arg){
    int sleepAmount = *(int*) arg;
    sleep(sleepAmount);
    printf("[Yawn] I slept for %d seconds\n",sleepAmount);
    return NULL;
}

int main(){
    int sleepArg1 = rand()%6 + 1;
    int sleepArg2 = rand()%7 + 2;
    pthread_create(&thread1,NULL,func1,&sleepArg1);
    pthread_create(&thread2,NULL,func1,&sleepArg2);
    ...
}
```

The threads dereference the pointer they were given. Note that the `sleepArg1` and `sleepArg2` variables are local to `main` but `main` doesn't return until after the joins, so they live long enough.

Mutexes

When two threads touch the same variable, you have a race. The canonical illustration is incrementing a shared counter:

`threadmutex.c`:

```
int ourCounter = 0;
pthread_mutex_t mutex;

void* threadCounter(void* arg){
    (void)arg;
    pthread_mutex_lock(&mutex);
    int temp = ourCounter;
    sleep(rand()%3+1);
    ourCounter = temp+1;
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main(){
    pthread_t threads[10];
    pthread_mutex_init(&mutex,NULL);
```

```

for(int i=0; i < 10; i++){
    pthread_create(&threads[i],NULL,threadCounter,NULL);
}
for(int i=0; i < 10; i++){
    pthread_join(threads[i],NULL);
}
pthread_mutex_destroy(&mutex);
printf("The value of ourCounter is: %d\n",ourCounter);
}

```

The `int temp = ourCounter; sleep(...); ourCounter = temp + 1` pattern is a deliberately bad idea — it widens the race window so much that without the mutex you’d get garbage almost every time. With the mutex, only one thread can be inside the `lock/unlock` block at a time, so the increments serialize and you get exactly 10.

The mutex lifecycle: `pthread_mutex_init` once at startup, `lock/unlock` pairs around critical sections, `pthread_mutex_destroy` once at the end.

10 Lecture 10 — Mutexes Reviewed and Condition Variables

Today we cleaned up the mutex example, then introduced condition variables, then ran a producer/consumer demo to see them in action.

Mutex review: bundling lock with the data it protects

`mutexreview.c` fixes a smell from last time: instead of one global mutex protecting one global counter, we bundle the mutex together with the counter into a struct.

```

struct protectedCounter{
    int counter;
    pthread_mutex_t mutex;
};

void* threadCounter(void* arg){
    struct protectedCounter* c = (struct protectedCounter*) arg;
    pthread_mutex_lock(&c->mutex);
    sleep(rand()%3+1);
    c->counter = c->counter+1;
    pthread_mutex_unlock(&c->mutex);
    return NULL;
}

```

Now you can have multiple independent counters, each with its own mutex, and the threading code doesn’t need to know which one it’s working on:

```

struct protectedCounter c1, c2;
c1.counter = 0; pthread_mutex_init(&c1.mutex,NULL);
c2.counter = 0; pthread_mutex_init(&c2.mutex,NULL);

for(int i=0; i < 10; i++){
    if(i % 2 == 1){
        pthread_create(&threads[i],NULL,threadCounter,&c1);
    }
}

```

```

} else {
    pthread_create(&threads[i], NULL, threadCounter, &c2);
}
}

```

This is the design pattern: *the mutex lives next to the data it protects, and they're passed together everywhere.*

Condition variables

A mutex protects shared state from concurrent modification. A *condition variable* lets a thread wait for shared state to reach some condition. The two are designed to work together:

- `pthread_cond_init` / `pthread_cond_destroy` — lifecycle.
- `pthread_cond_wait(&cond, &mutex)` — atomically unlock the mutex, sleep until signaled, re-acquire the mutex before returning.
- `pthread_cond_signal(&cond)` — wake one waiter.
- `pthread_cond_broadcast(&cond)` — wake all waiters.
- `pthread_cond_timedwait` — like wait, but with a timeout.

The trick that makes condition variables work is the *atomic* unlock-and-sleep in `wait`. Without it, you'd have a window between unlocking the mutex and going to sleep where a signal could arrive and be missed.

The bespoke producer/consumer

`condbespoke1.c` has two threads: a reader that asks for two numbers and an adder that sums them, signaling each other when input is ready.

```

pthread_mutex_t mut;
pthread_cond_t enoughInputs;
int numInputs = 0;
int inputArr[2];

void* adder(void* arg){
    pthread_mutex_lock(&mut);
    while(numInputs < 2){
        pthread_cond_wait(&enoughInputs, &mut);
    }
    int* num = (int*)arg;
    *num = inputArr[1] + inputArr[0];
    pthread_mutex_unlock(&mut);
    return NULL;
}

void* reader(void* arg){
    (void)arg;
    pthread_mutex_lock(&mut);
    while(numInputs < 2){
        printf("Enter a number:\n");
        scanf("%d", &inputArr[numInputs]);
    }
}

```

```

    numInputs++;
}
pthread_cond_signal(&enoughInputs);
pthread_mutex_unlock(&mut);
return NULL;
}

```

Two patterns to commit to memory:

- *Always wait in a loop: `while(numInputs < 2)` around the `pthread_cond_wait`, not `if`.* Spurious wakeups are allowed by the standard, and you might re-check after `wait` returns and find the condition isn't actually satisfied yet.
- *Always hold the mutex when checking the condition or calling `wait/signal`.* The mutex is what protects the variable that the condition is testing.

The classic producer/consumer

`condtest.c` is the textbook example with a bounded ring-buffer:

```

#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int count = 0, in = 0, out = 0;

pthread_mutex_t mutex;
pthread_cond_t cond_not_empty;
pthread_cond_t cond_not_full;

void* producer(void* arg){
    ...
    pthread_mutex_lock(&mutex);
    while (count == BUFFER_SIZE) {
        pthread_cond_wait(&cond_not_full, &mutex);
    }
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    count++;
    pthread_cond_signal(&cond_not_empty);
    pthread_mutex_unlock(&mutex);
    ...
}

void* consumer(void* arg){
    ...
    pthread_mutex_lock(&mutex);
    while (count == 0) {
        pthread_cond_wait(&cond_not_empty, &mutex);
    }
    int item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    count--;
    pthread_cond_signal(&cond_not_full);
    pthread_mutex_unlock(&mutex);
    ...
}

```

```
}
```

The producer blocks if the buffer is full, the consumer blocks if it's empty, and each one signals the other after every operation. Two condition variables, one mutex — the mutex protects all the shared state (`buffer`, `count`, `in`, `out`) and the conditions describe “the state has reached something interesting.”

We talked briefly about semaphores too. A semaphore is like a generalized mutex: it has a counter, and `sem_wait` decrements it (blocking if it's zero), `sem_post` increments it. With initial count 1, a semaphore behaves like a mutex; with higher counts you can let N threads through. Unlike mutexes, semaphores can be shared across processes when allocated in shared memory, which we'll see again at the end of the term.

11 Lecture 11 — File I/O in C

We're done with concurrency for now (mostly). Today: how to read and write files, both at the syscall level and through the `stdio.h` `FILE*` layer.

Reading a file with raw syscalls

`openFile1.c` uses `open/read/write` from `<unistd.h>` — the same syscalls we used in assembly:

```
int echoFile(char* fName, bool showSize){
    int fd = open(fName, O_RDONLY);
    char buffer[1024] = {0};
    ssize_t bytesRead = 0;

    if(showSize){
        struct stat fileStat;
        lstat(fName, &fileStat);
        printf("The file %s is %ld bytes long...\n", fName,
            (long)fileStat.st_size);
    }

    bytesRead = read(fd, buffer, sizeof(buffer));
    while(bytesRead > 0){
        write(STDOUT_FILENO, buffer, bytesRead);
        bytesRead = read(fd, buffer, sizeof(buffer));
    }
    close(fd);
    return 0;
}
```

The shape is: `open` returns a file descriptor (an `int`), `read` fills your buffer, `write` drains it, and you loop until `read` returns 0 (EOF) or a negative value (error). Then `close` the descriptor.

`lstat` (or `stat`) gives you metadata — size, permissions, modification time, etc. `lstat` differs from `stat` in how it handles symlinks: `lstat` returns info about the link itself, `stat` follows the link.

The driver dispatches on a `-s` flag:

```
if(strcmp(argv[1], "-s") == 0){
```

```

    echoFile(argv[2],true);
}
else{
    echoFile(argv[1],false);
}

```

The FILE* interface

openFile2.c uses the higher-level `stdio.h` API — `fopen` returns a `FILE*`, and you read line-by-line with `fgets`:

```

FILE* ourFile = fopen(argv[1],"r");
char** lines = malloc(fileSize * sizeof(char*));
int numLines = 0;
lines[0] = malloc(lineSize * sizeof(char));
while(fgets(lines[numLines],lineSize,ourFile)){
    numLines++;
    lines[numLines] = malloc(lineSize * sizeof(char));
}
free(lines[numLines]);

```

The pattern is “allocate a fresh buffer, read into it, advance the counter, allocate the next one” — you end up with one extra allocation when `fgets` returns `NULL`, which is why we `free(lines[numLines])` at the end.

Building a tiny line editor

lineEditor.c is the bigger payoff for the day — a `ed`-style text editor that loads a file, lets you insert / delete / edit lines, and writes the result back when you quit.

```

void cleanUp(FILE* f, char** ls, int nL){
    ftruncate(fileno(f),0);
    fseek(f, 0, SEEK_SET);
    for(int i=0; i < nL; i++){
        fputs(ls[i],f);
        free(ls[i]);
    }
    free(ls);
    fclose(f);
}

```

The `ftruncate(fileno(f),0) + fseek(f,0,SEEK.SET)` combo is how you wipe a file and rewrite it from scratch: `ftruncate` resizes the file to zero bytes, `fseek` puts the cursor back at the start. Then we just `fputs` each line back out.

```

void insLine(int line, char** ls, int* nL){
    char* newLine = malloc(linesize*sizeof(char));
    char c;
    printf("New text to insert at line %d:\n",line);
    while ((c = getchar()) != '\n' && c != EOF); // clear input buffer
    fgets(newLine, linesize, stdin);
    for(int i = *nL; i > line; i--){

```

```

    ls[i] = ls[i-1];
}
ls[line] = newLine;
(*nL)++;
}

```

The `while ((c = getchar()) != '\n' && c != EOF);` pattern is a recurring annoyance in C: after a `scanf("%d",&x)` reads a number, the trailing newline is still in the input buffer, and the next `fgets` would slurp just that newline and return immediately. So we eat the rest of the line first.

The main loop is a switch over a menu choice; nothing fancy, but it's a reminder that “a real program” is mostly bookkeeping around the small interesting bits. `foop.txt` in the directory is a sample file to experiment on.

12 Lecture 12 — Sockets, an Introduction

Today (no agenda file in the directory — I was running short on time) we wrote our way through the socket API for TCP, ending with a tiny one-page web server. The repo's `socketsGuide.org` covers the whole API in much more depth if you want to keep going.

The basic server pattern

The TCP server lifecycle is:

1. `socket(AF_INET, SOCK_STREAM, 0)` — get a socket file descriptor.
2. Fill in a `struct sockaddr_in` with family, address, port (port and address need to be in *network byte order* — use `htons` for the port, `INADDR_ANY` for “any address”).
3. `bind` the socket to that address.
4. `listen` on it (with a backlog size).
5. `accept` returns a fresh socket fd for each incoming connection.
6. `read/write` on that fd.
7. `close` the per-connection fd when done.

sock1.c: send a single greeting

```

int port = 8080;
int server_fd, new_socket;
struct sockaddr_in address;
int addrlen = sizeof(address);
char buffer[1024] = "Hey there y'all\n";

server_fd = socket(AF_INET, SOCK_STREAM, 0);
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(port);

bind(server_fd, (struct sockaddr*)&address, sizeof(address));

```



```
listen(server_fd, 3);

new_socket = accept(server_fd, (struct sockaddr*)&address,
                    (socklen_t*)&addrlen);
write(new_socket, buffer, strlen(buffer));

close(new_socket);
close(server_fd);
```

To test it, run the program in one terminal, then in another do `nc localhost 8080` (netcat). You'll see "Hey there y'all".

sock2.c: an echo server for one client

sock2.c adds a read/write loop so the server bounces whatever it gets back to the client:

```
new_socket = accept(server_fd, (struct sockaddr*)&address,
                    (socklen_t*)&addrlen);
while(1) {
    memset(buffer, 0, 1024);
    int valread = read(new_socket, buffer, 1024);
    if (valread > 0) {
        printf("Received: %s", buffer);
        write(new_socket, buffer, strlen(buffer));
    } else {
        break;
    }
}
```

This handles a single client; when they hang up, `read` returns 0 and we break out. The server then exits. Not super useful, but a step toward the real thing.

sock3.c: handle multiple clients sequentially

sock3.c wraps the accept/echo loop in an outer loop bounded by `maxClients = 5`:

```
while(clients < maxClients){
    new_socket = accept(server_fd, (struct sockaddr*)&address,
                        (socklen_t*)&addrlen);

    clients++;
    while(1) {
        memset(buffer, 0, 1024);
        int valread = read(new_socket, buffer, 1024);
        if (valread > 0) {
            printf("Received: %s", buffer);
            write(new_socket, buffer, strlen(buffer));
        } else {
            break;
        }
    }
}
```

This is sequential: the server services one client to completion before accepting the next. To handle multiple clients concurrently, you'd **fork** after **accept** (one process per client), or spawn a thread per client, or use **select** / **epoll** to multiplex on a single thread. We'll see those patterns in the next lecture and in the guide.

websock.c: a minimal HTTP server

The grand finale: a server that speaks HTTP/1.1 well enough to return “Hello, World” to a browser:

```
while (1) {
    memset(buffer,0,BUFFER_SIZE);
    new_socket = accept(server_fd, (struct sockaddr*)&address,
                        (socklen_t*)&addrlen);
    read(new_socket, buffer, BUFFER_SIZE);
    printf("Received request:\n%s\n", buffer);

    char *response = "HTTP/1.1 200 OK\r\n"
                     "Content-Type: text/html\r\n"
                     "Content-Length: 46\r\n"
                     "\r\n"
                     "<html><body><h1>Hello, World!</h1></body></html>";

    write(new_socket, response, strlen(response));
    close(new_socket);
}
```

HTTP is just lines of text over TCP. The status line, then headers, then a blank line (`\r\n\r\n`), then the body. The `Content-Length: 46` header is the byte count of the body — if you change the body, you have to update that number, otherwise the browser will hang waiting for bytes that aren't coming. Run this and visit `http://localhost:8080` in a browser; ignore the server log of the request that flies by.

13 Lecture 13 — IPC: Pipes, Named Pipes, and Shared Memory

The last regular lecture: a tour of the various ways processes can communicate without going through sockets. None of this is on an assignment, it's just fun.

Anonymous pipes

pipe1.c: a parent and child share a pipe, child writes, parent reads.

```
int pipefd[2]; // [0] read end, [1] write end
char write_msg[] = "Hello from the child process!";
char read_msg[100];

if(pipe(pipefd) == -1) { perror("Pipe failed"); return 1; }

pid_t pid = fork();

if (pid == 0) { // child
    close(pipefd[0]); // child doesn't read
    write(pipefd[1], write_msg, strlen(write_msg) + 1);
}
```

```

    close(pipefd[1]);
}
else { // parent
    close(pipefd[1]); // parent doesn't write
    read(pipefd[0], read_msg, sizeof(read_msg));
    printf("Parent received: %s\n", read_msg);
    close(pipefd[0]);
}

```

The pattern: `pipe()` fills in a 2-element array of fds, then you `fork`, then each side closes the end it doesn't need. Pipes are unidirectional — if you need bidirectional communication you make two pipes.

(**Caveat:** this code doesn't `wait()` for the child. Add a `wait(NULL)` at the end of the parent branch in real code.)

Two pipes, both ways: `tempconverter.c`

`tempconverter.c` demonstrates bidirectional communication: parent sends Fahrenheit values to child, child converts and sends back Celsius:

```

int parent_to_child[2]; // Parent writes, child reads
int child_to_parent[2]; // Child writes, parent reads

pipe(parent_to_child);
pipe(child_to_parent);

pid_t pid = fork();
if (pid > 0) { // parent
    close(parent_to_child[READ_END]);
    close(child_to_parent[WRITE_END]);
    for (int i = 0; i < count; i++) {
        sprintf(buffer, "%.1f", temperatures[i]);
        write(parent_to_child[WRITE_END], buffer, strlen(buffer)+1);
        read(child_to_parent[READ_END], buffer, BUFFER_SIZE);
        printf("Parent: Received result: %s\n", buffer);
    }
    strcpy(buffer, "EXIT");
    write(parent_to_child[WRITE_END], buffer, strlen(buffer)+1);
    wait(NULL);
}

```

Two pipes, each side closes the ends it doesn't use, and the parent sends a sentinel "EXIT" message to tell the child to clean up.

Named pipes (FIFOs)

A *named pipe* (a FIFO) is like a pipe with a path on the filesystem. Two unrelated processes can both open the path and talk to each other.

`namedPipesServer1.c` creates two FIFOs and reads from one / writes to the other:

```

mkfifo("clientToServer", 0666);

```

```

mkfifo("serverToClient", 0666);

fdWrite = open("serverToClient", O_WRONLY);
fdRead = open("clientToServer", O_RDONLY);

while(!stop_requested){
    memset(buffer, 0, bSize);
    int bytesRead = read(fdRead, buffer, bSize);
    if(bytesRead > 0){
        printf("Client: %s\n", buffer);
        write(fdWrite, buffer, bytesRead);
    }
    ...
}

```

mkfifo creates the file in the filesystem with the right type. Note the open order matters: open(O_WRONLY) blocks until someone opens the other end for reading. The whole client file is in `namedPipesClient1.c` — it spawns a reader thread to print whatever the server says, and reads stdin from the main thread.

The chat server/client (`chat_server.c` / `chat_client.c`) is a polished version of the same idea with proper signal-handling for cleanup.

Shared memory + semaphores

`shm_open` creates a chunk of memory that can be mapped into multiple processes' address spaces.

`sharedstruct.h` defines what they'll see:

```

struct sharedData {
    sem_t mutex;
    int payload1;
    int payload2;
};
#define sharedName "/shared_data_mem"

```

The semaphore lives *inside* the shared region so both processes can see it. `sharedMemprod.c`:

```

int sharedFd = shm_open(sharedName, O_CREAT | O_RDWR, 0666);
ftruncate(sharedFd, sizeof(struct sharedData));
struct sharedData* shared = mmap(NULL, sizeof(struct sharedData),
                                PROT_READ | PROT_WRITE,
                                MAP_SHARED, sharedFd, 0);
sem_init(&shared->mutex, 1, 1); // pshared=1 -> cross-process
for(int i = 0; i < 10; i++){
    sem_wait(&shared->mutex);
    shared->payload1 = i;
    shared->payload2 = i*i;
    sem_post(&shared->mutex);
    sleep(1);
}

```

The `sem_init(&shared->mutex, 1, 1)`'s middle argument is *pshared*: 1 means “shared between processes,” 0 means “shared between threads only.” This is the bit that lets one process wait on a semaphore another process posts to.

`sharedMemcons.c` is the consumer: it opens the same shared region (without `O_CREAT`), maps it, and reads from it under the same semaphore. At the end it calls `shm_unlink(sharedName)` to remove the kernel-resident object so a fresh run starts clean.

The bigger `shm_creator.c` / `shm_client.c` pair implements a collaborative drawing board: the creator displays a 20x10 character grid, and any number of clients can connect and randomly drop their character into the grid. Worth running with two or three clients in different terminals.

Message queues

`task_distributor.c` and `worker.c` use SysV message queues (`msgget`, `msgsnd`, `msgrcv`). The distributor sends task messages tagged `TASK_MSG = 1`; workers receive only that tag, do the (fake) work, and send back-tagged `RESULT_MSG = 2`. The queue acts as a typed, multi-consumer mailbox in the kernel. We didn't dig deeply into this one — it's old API and most modern code reaches for sockets or shared memory instead — but it's a useful tool to know exists.

Appendix — Struct Alignment and Optimization

This was a half-lecture I slotted in for some of the loose ends I'd been wanting to mention — struct field alignment, common compiler optimizations, and a quick aside about cache coherency. The files are in `lecture-inf/`.

Why does this struct take more space than I expected?

`structsize.c`:

```
struct Goofy1 { int num1; int num2; }; // 8 bytes
struct Goofy2 { int num1; double num2; }; // 16 bytes
struct Goofy3 { double num1; int num2; }; // 16 bytes
```

`Goofy1` is a clean 8 bytes, but `Goofy2` and `Goofy3` are both 16. The reason is *alignment*: hardware reads memory in chunks aligned to the architecture's word size, and to read a `double` the address needs to be a multiple of 8.

In `Goofy2`, the `int` takes 4 bytes; the `double` that follows it must start at offset 8, so the compiler inserts 4 bytes of padding between them.

In `Goofy3`, the `double` starts at offset 0 (fine), the `int` starts at offset 8 (4 bytes), and then the struct itself must be padded out to 16 so that arrays of `Goofy3` have each element properly aligned for its first `double`.

`structsize2.c` is a more elaborate example with multiple arrays:

```
struct exA { int int_field[3]; char char_field; long array[2]; };
struct exB { char char_field; long array[2]; int int_field[3]; };
struct exC { long array[2]; int int_field[3]; char char_field; };
```

The naive byte counts are all the same (29), but the actual `sizeofs` differ because of where the padding lands. The moral: *if you care about size, sort fields from largest to smallest.*

Loop unrolling

For tight loops where the body is short, the cost of the comparison and jump can be larger than the body itself. “Unrolling” means doing several iterations’ worth of work per loop pass:

```
// instead of:
for(int i = 0; i < 1000; i++){ arr[i] = i; }

// you might write (or the compiler might generate):
for(int i = 0; i < 1000; i += 5){
    arr[i] = i;
    arr[i+1] = i+1;
    arr[i+2] = i+2;
    arr[i+3] = i+3;
    arr[i+4] = i+4;
}
```

This pairs nicely with the CPU’s branch predictor and pipeline: fewer branches, more in-flight arithmetic. Don’t do this by hand unless you have a real reason to — the optimizer is better at deciding when it pays off than you are.

Function inlining

Calling a function costs a few instructions (push args, call, return, restore). For functions whose body is shorter than the call sequence, the compiler can substitute the body in place at the call site. The `inline` keyword is a hint; `static inline` and link-time optimization mostly take it from there.

Loop fusion

Two loops over the same range can be combined into one:

```
// before:
for(int i=0; i < 1000; i++){ arr[i] = i*i; }
for(int i=0; i < 1000; i++){ printf("arr[%d] = %d", i, arr[i]); }

// after:
for(int i=0; i < 1000; i++){
    arr[i] = i*i;
    printf("arr[%d] = %d", i, arr[i]);
}
```

You save a loop’s worth of overhead, and you also keep `arr[i]` hot in cache between the write and the read.

Cache coherency

The biggest performance win on modern hardware is being kind to the cache. Consecutive accesses to nearby addresses are essentially free; random accesses across megabytes are agonizingly slow. The classic example is matrix iteration order:

```

int m[100][200];
int accum = 0;

// bad: column-major access of a row-major array
for(int j=0; j<200; j++){
    for(int i=0; i<100; i++){
        accum += m[i][j]*m[i][j];
    }
}

// good: row-major access of a row-major array
for(int i=0; i<100; i++){
    for(int j=0; j<200; j++){
        accum += m[i][j]*m[i][j];
    }
}

```

In C, `m[i][j]` and `m[i][j+1]` are adjacent in memory; `m[i][j]` and `m[i+1][j]` are 200 ints apart. The good version touches 200 contiguous ints before moving to the next row, so each cache line gets fully used. The bad version strides through memory and constantly evicts/reloads cache lines. On a big matrix this is easily a 10x performance difference.

That's all for the term — the assignments, posit tutorial, and guides should round things out. See you in office hours.